

Lectura de cartas y control por gestos para Blackjack con visión por computadora: de datos sintéticos a una mesa real

Cristhian Pettico, Rodolfo Di Chiazza, Julian Blanco
Carrera de Especialización en Inteligencia Artificial (CEIA)
Facultad de Ingeniería, Universidad de Buenos Aires
Buenos Aires, Argentina

Resumen—Presentamos un prototipo que permite jugar al Blackjack sobre una mesa real: una cámara cenital reconoce las cartas que se reparten y el jugador pide carta o se planta con gestos de la mano. Comparamos dos estrategias para reconocer el índice de la esquina de cada carta: un detector YOLO11 de 13 clases (A) y un detector de una clase seguido de una ResNet-18 (B). Ambas alcanzan $mAP50 \approx 0,99$ en el dataset sintético de entrenamiento, pero sobre fotos reales caen a 0,68–0,86. El ajuste fino con fotos de otro mazo no se transfiere, mientras que con 79 cuadros del escenario propio el recall de A sube de 0,66 a 0,96 (F1 0,96). Evaluamos la robustez ante rotación, oclusión, iluminación y fondo: la oclusión es la perturbación más dañina, la iluminación afecta mucho más en datos reales que en sintéticos y el fondo casi no influye. El sistema completo (emparejamiento geométrico de esquinas, seguimiento ByteTrack, gestos con MediaPipe y un motor de reglas) resolvió correctamente las 4 partidas reales registradas.

Index Terms—detección de objetos, YOLO11, brecha sintético-real, robustez, seguimiento, gestos de la mano, Blackjack

I. INTRODUCCIÓN

Las mesas de juego de los casinos se supervisan con cámaras para registrar las manos, detectar errores del crupier y conductas como el conteo de cartas. Automatizar esa lectura es un problema clásico de visión por computadora: Zutis y Hoey [1] monitorearon partidas de Blackjack con una cámara estéreo cenital usando descriptores clásicos, y trabajos recientes integran el reconocimiento de cartas con la recomendación de la estrategia básica [2] o entrenan detectores YOLO sobre datasets sintéticos y reales de cartas [3]. El problema es en apariencia simple —52 clases bien definidas—, pero en una mesa real las cartas aparecen giradas, superpuestas, tapadas por las manos y bajo iluminación variable.

El **objetivo** de este trabajo es entrenar un detector de cartas y **evaluar experimentalmente su desempeño** ante rotaciones, oclusiones, cambios de iluminación y de fondo, e integrarlo en una aplicación que permita jugar Blackjack en una mesa real. Como en Blackjack el palo no influye en el puntaje, el reconocimiento se limita a los 13 valores. Las contribuciones son:

- Una comparación entre un detector de un paso (A) y uno de dos etapas (B), en precisión y latencia, sobre datos sintéticos, un dataset real público y un dataset real propio.
- La cuantificación de la brecha sintético-real y de cuándo el ajuste fino con fotos reales la cierra.

- Un estudio de robustez ante 4 familias de perturbaciones y un análisis de las confusiones entre valores.
- Un sistema completo y reproducible —cartas, seguimiento, gestos y reglas— validado con partidas reales.

II. METODOLOGÍA

La Fig. 1 resume el sistema. Los modelos no detectan la carta entera sino el **índice de la esquina** (valor y palo impresos), que es lo único legible cuando las cartas se superponen.

II-A. Datos

Sintético. 10.100 imágenes de 512×512 (7.070 / 2.020 / 1.010) derivadas del dataset *Playing Cards* de Roboflow [4]: cartas generadas y pegadas sobre fondos texturados, con las etiquetas reducidas a 13 valores (A) o a una clase (B). Para la CNN de B se recortaron las cajas con un margen del 10 % (28.280 / 8.080 / 4.040 recortes). **Real público.** 98 fotos de 416×416 (69 / 18 / 11) de [3]: un solo palo (corazones) y un mazo con índices en las cuatro esquinas. **Real propio.** Para el escenario de la demo se grabaron 4 sesiones con la cámara del juego (un celular cenital) y se extrajeron 130 cuadros (79 / 21 / 30) que se pre-etiquetaron con el modelo A y se corrigieron a mano: 1.688 esquinas, con muchas cartas superpuestas. Los conjuntos se separaron *por sesión*, para que cuadros casi iguales no queden en entrenamiento y prueba.

II-B. Modelos

A (un paso): YOLO11s [5], [6] preentrenado en COCO, ajustado para 13 clases (640 px, 50 épocas). Se eligió YOLO11 por su balance entre precisión y latencia en CPU y porque integra seguimiento. **B (dos etapas):** el mismo YOLO11s con una sola clase localiza las esquinas, se recortan con el mismo margen y una ResNet-18 [7] preentrenada en ImageNet clasifica el valor (96×96 , sin volteos porque un índice espejado no es una carta válida). La hipótesis de B es que separar “dónde” de “qué” facilita cada tarea. **Ajuste fino con fotos reales:** en cada época se mezclan 400 imágenes sintéticas con el entrenamiento real repetido 5 veces, y el mejor modelo se elige con la validación real; el test real no se usa hasta la evaluación. Se ajustó por separado con el dataset público (*ft*) y con el propio (*propio*).

Blackjack · diagrama general del sistema

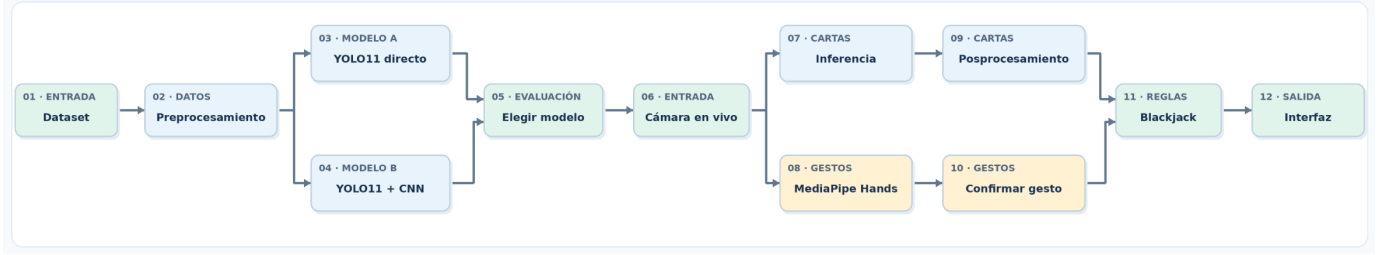


Figura 1. Arquitectura del sistema. Fuera de línea se entrenan los modelos A y B y se elige uno (bloques 01–05); en línea, cada cuadro de la cámara pasa por la inferencia de cartas y por MediaPipe, y ambas salidas alimentan el motor de reglas y la interfaz (bloques 06–12).

II-C. De esquinas a cartas y a la partida

Emparejamiento. Las dos esquinas de una carta están en vértices opuestos. Un par se acepta si la distancia entre centros, relativa al lado mayor de las cajas, está en $[3,3; 4,8]$ y el ángulo de la diagonal es cercano a 50° (carta vertical) o 140° (carta girada 90°). La ventana se calibró con las esquinas etiquetadas del dataset propio; usar el lado mayor y no la raíz del área la hace robusta a cartas giradas. Los pares se eligen de menor a mayor costo sin reutilizar esquinas, y una esquina sin pareja cuenta como carta (la otra puede estar tapada). Se descartan duplicados (supresión no máxima agnóstica a la clase, esquinas superpuestas del mismo valor) e índices “imposibles” dentro de una carta del mismo valor. **Seguimiento.** ByteTrack [8] asigna a cada esquina un identificador; su valor se decide por votación ponderada en 15 cuadros, una esquina perdida se recuerda 10 cuadros y la mano mostrada es la mediana, por valor, de los últimos 15 cuadros. **Gestos.** MediaPipe Hands [9] da 21 puntos por mano; la pose se clasifica con cocientes de distancias muñeca–punta / muñeca–nudillo, invariantes a la rotación y la escala: *apuntar* (solo el índice extendido, sostenido 0,5 s) pide carta y la *palma* abierta (sostenida 1,2 s o desplazada) se planta. **Reglas.** Un motor de estados (reparto, jugador, casa, fin) toma la mano estabilizada como fuente de verdad: la Casa pide hasta llegar a 17, el As vale 11 si no se pasa de 21, y un cambio se acepta tras 5 cuadros estables (20 si agrega cartas que nadie pidió). Mientras hay una mano sobre la mesa no se confirma ningún cambio.

II-D. Protocolo de evaluación

Una detección es correcta si su IoU con una esquina real es $\geq 0,5$ y el valor coincide. Se reportan mAP50 (promedio de los 13 valores), precisión (P), recall (R) y F1 con confianza $\geq 0,5$, y la latencia por imagen en CPU (AMD Ryzen 7 7840U). Para la robustez se aplican 21 perturbaciones a varios niveles (Fig. 2) —diseño adaptado de una versión previa del proyecto— y todas las configuraciones ven exactamente la misma imagen perturbada (semilla fija). Al rotar, las cajas reales se transforman junto con la imagen.

Tabla I
DESEMPEÑO POR ESQUINA ($\text{IoU} \geq 0,5$ Y VALOR CORRECTO) EN LOS TRES DOMINIOS. *ft*: AJUSTADO CON EL DATASET REAL PÚBLICO; *propio*: AJUSTADO CON EL DATASET PROPIO. LATENCIA EN CPU SOBRE EL TEST PROPIO.

Modelo	Sintético		Real público		Real propio	
	mAP50	F1	mAP50	F1	mAP50	F1
A base	0,995	1,00	0,68	0,72	0,86	0,78
B base	0,995 [†]	1,00	0,68	0,72	0,82	0,78
A <i>ft</i>	–	–	0,89	0,89	0,86	0,78
A <i>propio</i>	–	–	0,70	0,71	0,98	0,96
B <i>propio</i>	–	–	0,75	0,73	0,94	0,93

Latencia (ms/imagen): A 100–114; B 136–143

[†]Detector de B; la CNN alcanza 100 % de exactitud en el test sintético.

III. RESULTADOS

III-A. Brecha sintético–real y ajuste fino

La Tabla I muestra el resultado central. En el dominio sintético el problema está prácticamente resuelto (mAP50 0,995 y 100 % de exactitud de la CNN), pero el desempeño **cae a 0,68–0,86 de mAP50 en fotos reales**, una brecha conocida en el entrenamiento con datos sintéticos [10]. El ajuste fino la cierra, pero **solo en su propio dominio**: el modelo ajustado con el dataset público sube a F1 0,89 allí y queda igual que el base (0,78) en el escenario propio, y el ajustado con 79 cuadros propios llega a F1 0,96 en su escenario y queda igual que el base (0,71) en el público. Para un sistema desplegado, conviene invertir en pocas fotos del escenario real antes que en más datos genéricos.

En el escenario propio A supera a B (F1 0,96 frente a 0,93 ajustados) y es entre 20 y 40 ms más rápido; en el dataset público ambos quedan parejos (diferencias $\leq 0,02$). La ventaja de B se limita a una precisión ligeramente mayor del modelo base (0,99 frente a 0,96). Por eso la aplicación usa A ajustado con fotos propias.

III-B. Robustez

La Tabla II y la Fig. 3 resumen los experimentos sobre el test propio (30 cuadros, 412 esquinas) y una muestra del sintético (100 imágenes). **Oclusión.** Es la perturbación más dañina: en el sintético A tolera el 30 % del índice tapado (F1 0,95) y con

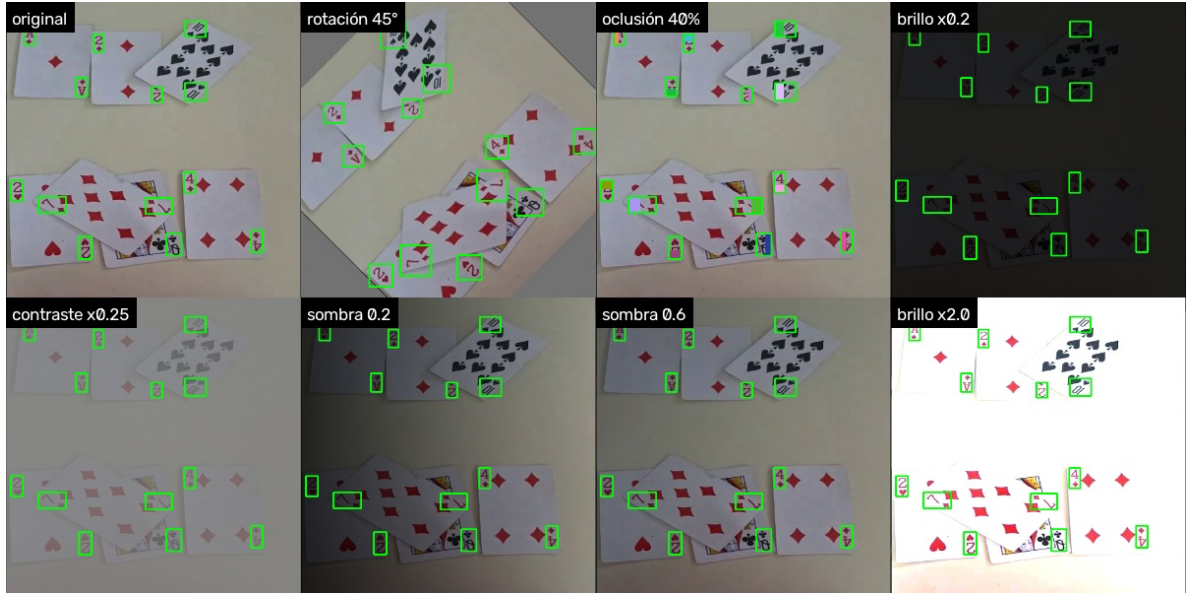


Figura 2. Ejemplos de perturbaciones sobre un cuadro del test propio, con las esquinas reales en verde: rotación, oclusión del índice (rectángulo de color desde un borde), brillo, contraste y sombra (gradiente de luz; el valor indica la luz mínima). Cada familia se evalúa en varios niveles de severidad (Fig. 3).

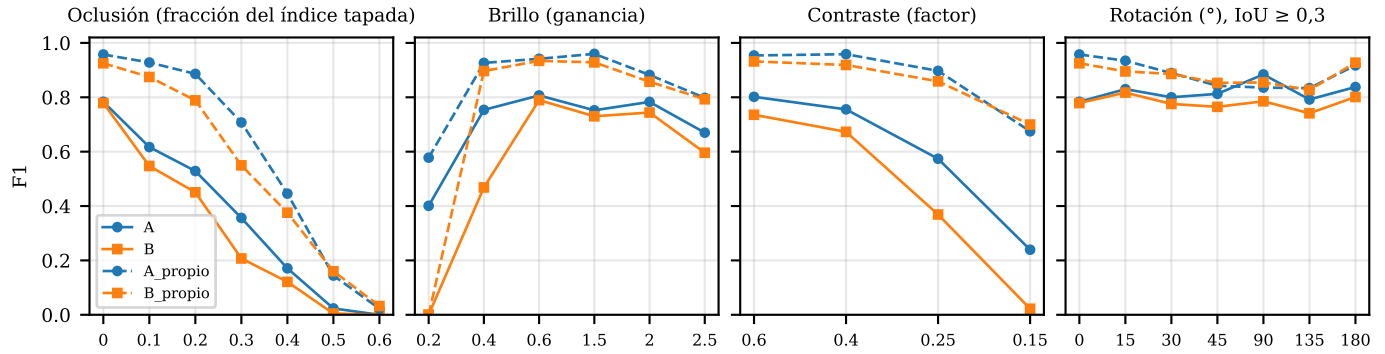


Figura 3. F1 en el test propio en función de la severidad de cada perturbación (trazo lleno: modelos base; trazo cortado: ajustados con fotos propias). Las rotaciones se miden con $\text{IoU} \geq 0,3$ (ver texto).

Tabla II
F1 MEDIO POR FAMILIA DE PERTURBACIÓN (PROMEDIO DE LOS NIVELES).

Dominio	Familia	A	B	A <i>propio</i>	B <i>propio</i>
Real propio	Rotación	0,76	0,72	0,81	0,81
	Oclusión	0,28	0,22	0,52	0,46
	Iluminación	0,69	0,58	0,88	0,82
Sintético	Rotación	0,74	0,72	0,74	0,71
	Oclusión	0,74	0,63	0,72	0,61
	Iluminación	0,99	0,98	0,99	0,98

el 50 % cae a 0,52; en datos reales, donde las mesas ya tienen cartas superpuestas, el modelo base pierde desde el 10 % (0,62) y el ajustado resiste hasta el 20 % (0,89). Esto motivó contar como carta una esquina suelta, la memoria del seguimiento y congelar la mesa mientras hay una mano encima. **Iluminación.** En el sintético los modelos son casi inmunes ($F1 \geq 0,98$), porque ese dataset ya incluye variaciones de brillo; en datos

reales la subexposición y el bajo contraste son críticos: con brillo $\times 0,2$ B obtiene F1 0 y A ajustado 0,58, y con contraste $\times 0,15$ A base cae a 0,24 y el ajustado mantiene 0,67. Las sombras casi no afectan. El ajuste fino mejora la robustez a la iluminación en +0,19 de F1 medio. **Rotación.** Con $\text{IoU} \geq 0,5$ el F1 cae a 0,46 a 45° incluso en el sintético, pero es un efecto de la medición: la caja real rotada es la caja alineada que contiene al índice girado, más grande que la predicción. Con $\text{IoU} \geq 0,3$ el sintético da $F1 \geq 0,96$ en todos los ángulos; en datos reales queda una caída genuina (de 0,96 a 0,84 a 45°), mientras que 90° y 180° se resuelven bien. **Fondo.** Reemplazar el fondo de una foto con una máscara de color no resultó confiable, por lo que se compusieron 30 escenas con 7 cartas reales recortadas del mazo de la demo sobre 8 fondos (madera, paño verde, oscuro, blanco, textura, desorden de colores, ruido y mármol), con las mismas cartas y posiciones en todos. A obtiene $F1 \geq 0,99$ en los 8; el único fondo que afecta es el blanco para B base (0,90), cuyo detector encuentra el 83 % de

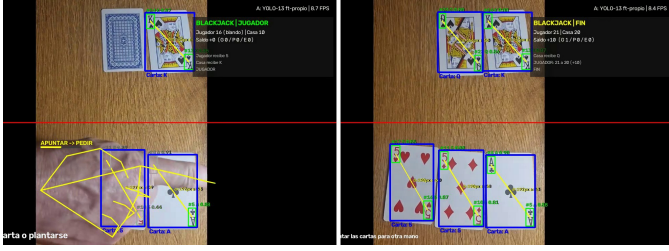


Figura 4. Partida real. Izquierda: el gesto *apuntar* pide carta ($A + 5 = 16$). Derecha: el jugador llega a 21 con un 5 girado y gana 21 a 20 (panel: etapa, puntajes, saldo y eventos).

las esquinas. Con cartas separadas, el fondo no es la dificultad: lo son la superposición y la oclusión.

III-C. Confusiones

En el test propio casi no se confunden valores: A base acierta el 66,3 % de las esquinas, se equivoca de valor en el 1,7 % y **no encuentra el 32,0 %**; ajustado acierta el 95,6 %, no confunde ninguna y no encuentra el 4,4 %. Las confusiones que aparecen son entre formas parecidas ($8 \rightarrow 6$ en A base, $5 \rightarrow 3$ y $8 \rightarrow Q$ en B ajustado, que clasifica un recorte sin contexto), y el 8 es el valor más difícil de localizar (13 de las 18 esquinas perdidas).

III-D. Sistema completo

Seguimiento. En 4 videos con movimiento de cámara simulado, el seguimiento elevó la mano mostrada correcta de 50,5 % a 76,2 % y redujo los cambios de mano en 10 s de 47 a 4. **Gestos.** Sobre fotos reales de HaGRID [11], la regla propuesta reconoce *apuntar* y *palma* con recall 0,80 y 0,89 tanto a 0° como a 90° y 180° , mientras que el reconocedor de gestos de MediaPipe cae a 0 con la mano girada 90° o 180° , situación habitual con una cámara cenital. En clips de video, *palma* se reconoce en el 82–87 % de los casos con 4,2 % de disparos falsos. **Partidas reales.** Se grabaron 4 partidas completas (2 ganadas por el jugador y 2 por la Casa, incluyendo un As que pasa de 11 a 1 y una carta girada). Reproducidas cuadro a cuadro, el sistema obtiene en las 4 el mismo resultado, cartas y gestos que se vieron en vivo, sin irregularidades (Fig. 4); el prototipo corre a 7–10 cuadros por segundo en CPU. Estas partidas quedan como prueba de regresión del sistema.

IV. CONCLUSIONES

1. El rendimiento en el dataset sintético (mAP50 0,995) no anticipa el rendimiento real (0,68–0,86): evaluar con fotos del escenario de uso es imprescindible.
2. El ajuste fino con datos reales solo mejora el dominio con el que se hace; 79 cuadros del escenario propio llevaron el recall de 0,66 a 0,96, mientras que fotos de otro mazo no aportaron nada.
3. En el escenario de uso, el detector de un paso (A) superó al de dos etapas (B) en exactitud y latencia: el contexto que ve el detector ayuda a distinguir valores parecidos, mientras que la CNN de B clasifica un recorte aislado.

4. En datos reales el error dominante no es confundir valores sino no encontrar la esquina; la oclusión es la perturbación más dañina y la iluminación pesa mucho más que en los datos sintéticos.
5. El fondo casi no influye, y la caída aparente ante rotaciones oblicuas se explica en gran parte por la métrica (IoU de cajas alineadas); conviene reportar ambos umbrales.
6. La lógica de posprocesamiento —emparejamiento geométrico, NMS agnóstico, seguimiento y congelar la mesa con una mano encima— fue decisiva para pasar de detecciones por cuadro a manos correctas en partidas reales.
7. Para gestos con cámara cenital, reglas invariantes a la rotación sobre los puntos de MediaPipe funcionan donde el clasificador preentrenado falla.

Trabajo futuro: reconocer el palo (permitiría detectar cartas duplicadas, imposibles con un solo mazo), entrenar con la cámara en ambas orientaciones, más mesas e iluminaciones en el test propio y varios jugadores. El código, los modelos, los datos y las partidas reales están disponibles en https://github.com/dichiazzar/Vision_Computadora_2_TP_2.

REFERENCIAS

- [1] K. Zutis and J. Hoey, “Who’s counting? Real-time blackjack monitoring for card counting detection,” in *Proc. 7th Int. Conf. on Computer Vision Systems (ICVS)*, ser. Lecture Notes in Computer Science, vol. 5815. Liège, Belgium: Springer, 2009.
- [2] K. Gupta, D. Bolt, and B. Hinchliff, “Optimal blackjack strategy recommender: A comprehensive study on computer vision integration for enhanced gameplay,” arXiv:2404.00191, 2024.
- [3] T. Kostadinov, “Playing cards object detection,” <https://github.com/TeogopK/Playing-Cards-Object-Detection>, 2024, proyecto del curso Deep Learning, FMI, Universidad de Sofía; dataset real bajo licencia CC0.
- [4] Augmented Startups, “Playing cards dataset,” Roboflow Universe, <https://universe.roboflow.com/augmented-startups/playing-cards-ow27d>, accedido: oct. 2026.
- [5] G. Jocher and J. Qiu, “Ultralytics YOLO11,” <https://github.com/ultralytics/ultralytics>, 2024, versión 11.0.0, licencia AGPL-3.0.
- [6] R. Khanam and M. Hussain, “YOLOv11: An overview of the key architectural enhancements,” arXiv:2410.17725, 2024.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [8] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-object tracking by associating every detection box,” in *Proc. European Conf. on Computer Vision (ECCV)*, 2022.
- [9] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, “MediaPipe Hands: On-device real-time hand tracking,” arXiv:2006.10214, 2020.
- [10] J. Tremblay, A. Prakash, D. Acuna, M. Brophy, V. Jampani, C. Anil, T. To, E. Cameracci, S. Bochoon, and S. Birchfield, “Training deep networks with synthetic data: Bridging the reality gap by domain randomization,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018, pp. 969–977.
- [11] A. Kapitanov, K. Kvanchiani, A. Nagaev, R. Kraynov, and A. Makhliarchuk, “HaGRID – HAnd Gesture Recognition Image Dataset,” in *Proc. IEEE/CVF Winter Conf. on Applications of Computer Vision (WACV)*, 2024.